

Exame Modelo 1 — Paradigmas de Programação — Época Normal / Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época Normal / Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
 - Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.
 - É estritamente proibida a utilização de coleções da biblioteca `java.util`.
-

Parte 1 (6 Valores)

Pergunta 1 (1,5 valores)

Explique detalhadamente as diferenças entre classes abstratas e interfaces em Java. Em que situações é mais adequado optar por uma classe abstrata e em que situações é preferível uma interface? Justifique a sua resposta e ilustre cada caso com um exemplo prático que evidencie as características distintivas de cada mecanismo.

Pergunta 2 (1,5 valores)

Descreva detalhadamente o modo como Java realiza a passagem de argumentos para os métodos, distinguindo o comportamento aplicado a tipos primitivos do comportamento aplicado a referências de objetos. Esclareça os equívocos frequentes associados a este mecanismo e fundamente a sua explicação com exemplos concretos que demonstrem os efeitos sobre os valores e os estados dos objetos.

Pergunta 3 (1,5 valores)

Explique o conceito de conversão de tipos (*casting*) no contexto da herança e do polimorfismo. Discuta os riscos associados a conversões incorretas e as situações em que cada tipo de conversão é apropriado. Ilustre com um exemplo prático que demonstre as conversões seguras com o operador `instanceof`.

Pergunta 4 (1,5 valores)

Distinga os conceitos de identidade e de igualdade de objetos em Java, esclarecendo a diferença entre o operador `==` e o método `equals()`. Explique igualmente o papel do método `toString()`. Forneça um exemplo prático que demonstre a redefinição correta dos métodos `equals()` e `toString()` numa classe.

Parte 2 (14 Valores)

1. Considere as seguintes interfaces `Vehicle` e `RefrigeratedVehicle`. A interface `Vehicle` descreve as operações possíveis que definem o contrato de um veículo utilizado na recolha de bens de uma instituição de ajuda humanitária. A interface `RefrigeratedVehicle` especializa `Vehicle` e acrescenta operações específicas para veículos refrigerados com restrições de distância.

```
public interface Vehicle {  
    String getCode();  
    ItemType getSupplyType();  
    double getMaxCapacity();  
}
```

```
public interface RefrigeratedVehicle extends Vehicle {  
    double getMaxKilometers();  
    boolean equals(Object obj);  
}
```

Pergunta 1a (3 valores)

Considere a interface `RefrigeratedVehicle` que representa um veículo refrigerado com um limite máximo de quilómetros. Implemente a interface numa classe denominada `RefrigeratedVehicleImpl`. O veículo deve possuir um **estado** (`enabled`, booleano) e deve ser inicializado como `true` (habilitado) por defeito no construtor. Para a implementação do método `equals`, considere que **duas instâncias** de `RefrigeratedVehicle` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`).

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

2. Considere a existência de uma classe `StrategyImpl` que implemente a interface `Strategy`.

```
public interface Strategy {  
    Route[] generate(IIInstitution inst, RouteValidator validator);  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `StrategyImpl` que podem ser utilizados para a geração da rota:

```
boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox);
```

- Este método deve devolver `true` caso exista a ocorrência de, pelo menos, um contentor cujo tipo seja igual ao do veículo e se a sua última medição registada tiver um valor superior a 80% da sua capacidade.

```
boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator);
```

- O método deve devolver `true` caso a `AidBox` seja adicionada à rota.
- Para adicionar a `AidBox` à rota, deve previamente efetuar uma validação com recurso ao método `validator.validate(route, aidbox)`:
 - Se validado, a `AidBox` deve ser adicionada à rota utilizando o método `void addAidBox(AidBox aidBox)` da classe `Route`.
 - Caso a invocação do método `addAidBox` origine uma `RouteException`, o método `addAidBoxToRoute` deve retornar `false`.

Pergunta 2b (5 valores)

Na classe `StrategyImpl`, implemente o método `generate`, gerando as rotas necessárias.

Regras a considerar:

- Para cada veículo devolvido pelo método `getVehicles()` da interface `IIInstitution`, deve ser criada uma nova rota. Assuma que só existe um veículo para cada tipo.
- As `AidBoxes` existentes são as devolvidas pelo método `getAidBoxes()` da interface `IIInstitution`.
- Deve utilizar os métodos desenvolvidos na alínea anterior. Se não os implementou anteriormente, assuma que os métodos já existem.
- O array devolvido pelo método `generate` deve conter rotas com as `AidBoxes` (sem posições nulas ou rotas vazias).

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IIInstitution {  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;
```

```
private Vehicle[] vehicles;
private int numberOfVehicles;

public Vehicle[] getVehicles() { return this.vehicles; }
public AidBox[] getAidBoxes() { return this.aidBoxes; }
}
```

```
public interface AidBox {
    Container[] getContainers();
    String getCode();
}
```

```
public interface Container {
    ItemType getType();
    double getCapacity();
    Measurement getLastMeasurement();
}
```

```
public interface Measurement {
    double getValue();
}
```

```
public interface Route {
    void addAidBox(AidBox aidBox) throws RouteException;
    AidBox removeAidBox(AidBox aidBox) throws RouteException;
    AidBox[] getRoute();
}
```